SỞ GIÁO DỤC VÀ ĐÀO TẠO
TỈNH LÂM ĐỒNG
ĐỀ THI THỬ SỐ 07

KỲ THI CHỌN HỌC SINH GIỎI CẤP TỈNH
LỚP 12 THPT
Môn thi: TIN HỌC (Lập trình Python)
Thời gian: 90 phút (không kể phát đề)

DANH MỤC CÁC BÀI THI

Câu	Tên bài	File mã nguồn	File input	File output	Điểm
Câu 1	PRIMEODD	PRIMEODD.PY	PRIMEODD.INP	PRIMEODD.OUT	10.0
Câu 2	SAURIENG	SAURIENG.PY	SAURIENG.INP	SAURIENG.OUT	10.0

Câu 1 (10.0 điểm): BÓC TÁCH SỐ NGUYÊN TỐ CHỮ SỐ LẺ (PRIMEODD)

Trong đợt mã hóa dữ liệu văn bản nông nghiệp, hệ thống ghi nhận các xâu ký tự hỗn hợp chứa cả chữ cái và chữ số. Các kỹ sư muốn lọc ra các số nguyên có tính chất toán học đặc biệt để làm khóa mã hóa.
Cho một xâu ký tự S có độ dài không quá 10^5 ký tự. Các chữ số liên tiếp trong xâu S tạo thành một số nguyên dương. Một số nguyên X bóc tách từ xâu S được gọi là **Số nguyên tố chữ số lẻ** nếu thỏa mãn đồng thời hai điều kiện:

- X là một số nguyên tố.
- Tất cả các chữ số cấu thành nên X đều là chữ số lẻ (1, 3, 5, 7, 9).

Yêu cầu: Đếm số lượng số nguyên tố chữ số lẻ có trong xâu S, in ra danh sách các số đó theo thứ tự xuất hiện và tính tổng của chúng.

Dữ liệu vào: Đọc từ tệp **PRIMEODD.INP** gồm một dòng duy nhất chứa xâu S.
Dữ liệu ra: Ghi ra tệp **PRIMEODD.OUT** theo cấu trúc:

- Dòng 1: Ghi số lượng số nguyên tố chữ số lẻ tìm được (nếu không có số nào thì ghi -1).
- Dòng 2: Ghi danh sách các số tìm được, mỗi số cách nhau một khoảng trắng.
- Dòng 3: Ghi tổng của tất cả các số tìm được.

PRIMEODD.INP	PRIMEODD.OUT
abc37xyz23def13k79m	3 37 13 79 129

Giải thích: Các số bóc tách được là 37, 23, 13, 79. Số 23 là số nguyên tố nhưng chứa chữ số 2 chẵn nên loại. Còn lại 37, 13, 79 đều là số nguyên tố và toàn chữ số lẻ. Tổng = 37 + 13 + 79 = 129.

Ràng buộc Subtask:
- Subtask 1 (60% số điểm): Xâu S có độ dài N <= 1000, các số bóc tách được <= 10^4.
- Subtask 2 (40% số điểm): Xâu S có độ dài N <= 10^5, các số bóc tách được <= 10^9.

Câu 2 (10.0 điểm): THU HOẠCH SẦU RIÊNG ĐẠ TẠI (SAURIENG)

Huyện Đạ Tại (tỉnh Lâm Đồng) là địa danh nổi tiếng với vùng trồng sầu riêng chất lượng cao. Chuẩn bị vào vụ thu hoạch, một hợp tác xã nông nghiệp tập hợp danh sách N nhà vườn có sầu riêng chín tới, nhà vườn thứ i có sản lượng dự kiến thu hoạch là A[i] (kg).
Một doanh nghiệp đưa xe tải có tải trọng tối đa K (kg) đến thu mua. Để giúp đỡ được **nhiều hộ nông dân nhất** (thu mua từ nhiều nhà vườn nhất có thể), doanh nghiệp quyết định áp dụng chiến thuật: Ưu tiên thu mua hết sản lượng của các nhà vườn có sản lượng nhỏ trước.

Yêu cầu: Hãy xác định số lượng nhà vườn nhiều nhất mà xe tải có thể thu mua hết sản lượng và tổng khối lượng sầu riêng thực tế xe đã bốc xếp lên xe.

Dữ liệu vào: Đọc từ tệp **SAURIENG.INP** gồm:

- Dòng 1: Ghi hai số nguyên dương N và K (N <= 10^5, K <= 10^12) cách nhau một khoảng trắng.
- Dòng 2: Ghi N số nguyên dương A[1], A[2], ..., A[N] (1 <= A[i] <= 10^9) là sản lượng sầu riêng của từng nhà vườn.

Dữ liệu ra: Ghi ra tệp **SAURIENG.OUT** gồm một dòng duy nhất ghi hai số nguyên cách nhau một khoảng trắng: số lượng nhà vườn nhiều nhất được thu mua và tổng khối lượng sầu riêng thực tế chở được.

SAURIENG.INP	SAURIENG.OUT
5 1000 300 150 200 350 500	4 1000

Giải thích: Sản lượng sau khi sắp xếp tăng dần: 150, 200, 300, 350, 500. Thu mua lần lượt: 150 + 200 + 300 + 350 = 1000 kg (đạt đúng tải trọng K=1000). Thu mua được 4 nhà vườn.

Ràng buộc Subtask:
- Subtask 1 (50% số điểm): N <= 1000, A[i] <= 10^5.
- Subtask 2 (50% số điểm): N <= 10^5, A[i] <= 10^9, K <= 10^12.

ĐÁP ÁN VÀ HƯỚNG DẪN CHẤM ĐỀ THI THỬ SỐ 07

CÂU 1: PRIMEODD (10.0 ĐIỂM) - PHÂN TÍCH THUẬT TOÁN & CODE MẪU

- Phân tích giải thuật:

+ Bước 1: Duyệt từng ký tự trong xâu S. Nếu là chữ số (char.isdigit()), gom nối tiếp vào biến tạm temp. Ngược lại, nếu gặp ký tự chữ cái thì ép kiểu int(temp) cất vào danh sách số.

+ Bước 2: Đừng quên kiểm tra biến temp ngoài vòng lặp để không bị bỏ sót số cuối cùng của xâu.

+ Bước 3: Cài đặt hàm is_prime(n) kiểm tra số nguyên tố tối ưu O(sqrt(n)). Cài đặt hàm check_all_odd_digits(n) kiểm tra xem mọi chữ số trong n có phải số lẻ (1, 3, 5, 7, 9) hay không.

+ Bước 4: Lọc các số thỏa mãn, đếm số lượng, in danh sách và tính tổng ghi ra file PRIMEODD.OUT.

Mã nguồn Python minh họa (PRIMEODD.PY):

```
import math

def is_prime(n):
    if n < 2: return False
    if n in (2, 3): return True
    if n % 2 == 0 or n % 3 == 0: return False
    for i in range(5, int(math.isqrt(n)) + 1, 6):
        if n % i == 0 or n % (i + 2) == 0:
            return False
    return True

def all_odd_digits(n):
    for digit in str(n):
        if int(digit) % 2 == 0:
            return False
    return True

# Đọc dữ liệu từ file PRIMEODD.INP
with open("PRIMEODD.INP", "r", encoding="utf-8") as f:
    s = f.read().strip()

# Bóc tách các số nguyên từ xâu S
temp = ""
numbers = []
for char in s:
    if char.isdigit():
        temp += char
    else:
        if temp != "":
            numbers.append(int(temp))
            temp = ""
if temp != "":
    numbers.append(int(temp))

# Lọc các số thỏa mãn cả hai điều kiện
result_numbers = [x for x in numbers if is_prime(x) and all_odd_digits(x)]
```

```
# Ghi kết quả ra file PRIMEODD.OUT
with open("PRIMEODD.OUT", "w", encoding="utf-8") as f:
    if not result_numbers:
        f.write("-1\n")
    else:
        f.write(f"{len(result_numbers)}\n")
        f.write(" ".join(map(str, result_numbers)) + "\n")
        f.write(f"{sum(result_numbers)}\n")
```

CÂU 2: SAURIENG (10.0 ĐIỂM) - PHÂN TÍCH THUẬT TOÁN & CODE MẪU

- Phân tích giải thuật (Thuật toán Tham lam - Greedy):
  + Yêu cầu thu mua từ NHIỀU NHÀ VƯỜN NHẤT dẫn đến chiến lược Tham lam: Chọn các nhà vườn có sản lượng nhỏ nhất trước.
  + Sắp xếp mảng sản lượng A tăng dần bằng phương thức a.sort() có độ phức tạp O(N log N).
  + Duyệt từng sản lượng đã sắp xếp, cộng dồn vào tổng khối lượng total_weight. Nếu total_weight + price <= K thì tiếp tục thu mua và tăng biến đếm count += 1. Khi tổng vượt quá K thì dừng lại.
  + Ghi số lượng count và tổng khối lượng total_weight ra file SAURIENG.OUT.

Mã nguồn Python minh họa (SAURIENG.PY):

```
# Đọc dữ liệu từ tệp SAURIENG.INP
with open("SAURIENG.INP", "r", encoding="utf-8") as f:
    lines = f.read().split()

if lines:
    n = int(lines[0])
    k = int(lines[1])
    a = [int(x) for x in lines[2:]]

    # Chiến thuật tham lam: Sắp xếp sản lượng tăng dần
    a.sort()

    count = 0
    total_weight = 0

    for weight in a:
        if total_weight + weight <= k:
            total_weight += weight
            count += 1
        else:
            break

    # Ghi kết quả ra tệp SAURIENG.OUT
    with open("SAURIENG.OUT", "w", encoding="utf-8") as f:
        f.write(f"{count} {total_weight}\n")
```